arXiv:2506.06295v3 [cs.LG] 6 Jun 2026

dLLM-Cache：通过自适应缓存加速扩散大语言模型

Zhiyuan Liu [*1 2] **Yicun Yang** [*1 2] **Yaojie Zhang** [2 3] **Junjie Chen** [2] **Chang Zou** [1 2 3] **Qingyan Wei** [2]
Shaobo Wang [1 2] **Yichen Zhu** [4] **Linfeng Zhang** [1 2]

摘要

自回归模型（ARM）长期主导大语言模型领域。近年来，基于扩散的大语言模型（dLLM）作为一种新范式出现，它通过对掩码片段迭代去噪来生成文本，展现出显著优势与潜力。然而，dLLM 的推理延迟较高。由于采用双向注意力机制，键值缓存等传统 ARM 加速技术无法直接兼容 dLLM。为应对这一挑战，我们从一个关键观察出发：dLLM 推理包含静态提示与部分动态响应，其中大多数 token 在相邻去噪步之间保持稳定。基于此，我们提出 dLLM-Cache：一种无需训练的自适应缓存框架，将长间隔提示缓存与由特征相似度引导的响应局部更新相结合。这一设计能够高效复用中间计算，同时不损害模型性能。在 LLaDA 8B、Dream 7B 等代表性 dLLM 上的广泛实验表明，dLLM-Cache 在 LongBench-HotpotQA 上最多可将 FLOPs 降低至原来的 1/9.1，同时保持有竞争力的输出质量。值得注意的是，在许多设置下，我们的方法使 dLLM 的推理延迟接近 ARM。代码公开于：https://github.com/maomaocun/dLLM-cache。

双向导致之前存的值是过期的，因为上下文的改变

1. 引言

大语言模型（LLM）（Zhao et al., 2023）是现代人工智能的基础，支撑着从对话式人工智能到科学发现的各种应用。尽管自回归模型（ARM）一直占据主导地位（Radford et al., 2018; Brown et al., 2020; OpenAI, 2022），LLaDA（Nie et al., 2026）和 Dream（Ye et al., 2025）等基于扩散的大语言模型（dLLM）已成为有前景的替代方案。这些模型具有出色的可扩展性；凭借双向注意力机制，它们在“逆向诅咒”（Berglund et al., 2024）等挑战上优于 ARM，展现出扩散模型处理复杂语言任务的潜力。

dLLM 的实际应用受到一个矛盾的制约：尽管具备并行解码潜力，其计算复杂度却高达 $O(N^3)$。低效的原因在于，实践中生成长度为 N 的序列需要 N 次去噪迭代，每次都在没有缓存机制的情况下重新计算所有 token 的双向注意力。这从根本上低于标准 ARM 的效率；后者利用键值缓存（Pope et al., 2023），可将整体计算量降至 $O(N^2)$。

我们通过将缓存机制成功应用于 dLLM，弥合这一差距。首先，如图 1 所示，我们研究了 dLLM 推理中两种统一策略无法解决的计算冗余。其一，**提示冗余**：输入提示 token 保持不变，但其内部表示（例如注意力输出）仍在每个去噪步重新计算。其二，**响应的动态性与冗余**：生成响应的特征随时间演化。相邻步骤通常存在显著相似性，表明有缓存潜力，但并非所有 token 都以相同方式演化。这种非均匀演化解释了传统统一缓存策略为何无效。

response里面变化的占少数

受这些认识启发，我们提出 dLLM-Cache，一种无需训练的自适应缓存机制，通过利用这些不同类型的冗余来加速 dLLM 推理。dLLM-Cache 采用差异化缓存策略，包含两个核心组成部分：

* 共同贡献。[1]上海交通大学人工智能学院，中国上海；[2]上海交通大学 EPIC 实验室，中国上海；[3]电子科技大学，中国成都；[4]美的集团，中国。通信作者：Linfeng Zhang <zhanglinfeng@sjtu.edu.cn>。

第 43 届国际机器学习大会论文集，韩国首尔。PMLR 306，2026。版权归作者所有，2026 年。

Chinese translation | original equations, data and references retained

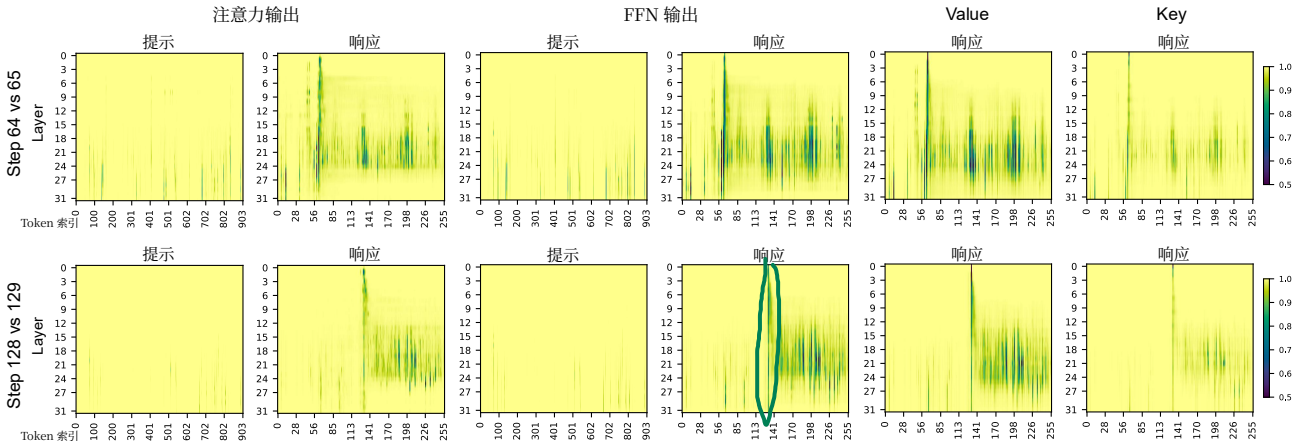


图 1. dLLM 相邻两个去噪步之间 Key、Value、注意力输出与 FFN 输出的余弦相似度，揭示计算冗余。热力图分别展示提示与响应 token 在相邻步骤间的相似性，颜色越浅，表示该 token 相对于上一步的值越相似。结果表明：（I）提示区域相似性较高，而响应区域中不同 token 的相似性不同；（II）只有少量响应 token 的相似性显著较低，说明选择性重计算已足够；（III）响应 token 的 Value 相似性与注意力、FFN 输出的相似性高度一致，支持以 Value 变化识别变化最大的响应 token。

深色线条

prompt极慢刷新

· **长间隔提示缓存：**仅按稀疏的长间隔（例如每 100 步）计算并缓存提示 token 的相关特征。在下一个长间隔刷新点之前，所有中间步骤均复用这些缓存特征，从而大幅降低处理静态提示的开销。

· **自适应短间隔响应缓存：**对响应 token 的特征进行缓存，并按更频繁的短间隔（例如每 10 步）完整刷新。在两次完整刷新之间，采用自适应局部更新策略以平衡速度与准确率。具体而言，我们识别并只更新动态性最强的 token。如图 1 所示，token 的 Value 向量在相邻步骤间的余弦相似度，与其后续注意力和 FFN 输出的变化高度相关。这促成了我们的 V-verify 机制：以 Value 相似度为高效代理，选择需要更新的 token。

更短间隔内就要全部刷新一遍

这种针对提示与响应特征的差异化、自适应处理，无需重新训练便能显著加速推理并保持质量。主要贡献如下：

1. 识别并刻画 dLLM 推理中的双重计算冗余：近乎静态的提示冗余，以及动态响应冗余。

2. 提出无需训练的自适应缓存框架 dLLM-Cache，将长间隔提示缓存与短间隔、相似度引导的响应 token 局部更新相结合。

3. 提出轻量而有效的 V-verify 机制，利用 Value 向量跨去噪步的余弦相似度，识别变化最大的 token 并局部更新；其依据是该指标与 token 整体演化之间较强的实证相关性。

4. 在多种基准上验证 dLLM-Cache，实现显著推理加速，例如 LLaDA 上最高 9.1 倍的加速，同时保持有竞争力的输出质量；相较基线及更简单的缓存方法，取得更优的速度—质量折中。

2. 相关工作

2.1. 用于语言的扩散模型

扩散模型（DM）（Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021）学习逆转数据损坏过程，在图像等连续领域表现出色（Rombach et al., 2022; Peebles & Xie, 2023）。但将其用于文本等离散数据仍有挑战，部分原因在于文本的离散性质。一个有前景的方向是掩码扩散模型（MDM）（Austin et al., 2021; Lou et al., 2024; Shi et al., 2024; Ou et al., 2025; Zheng et al., 2024; Gong et al., 2025; Nie et al., 2025; He et al., 2023; Reid et al., 2023; Sahoo et al., 2024; Ye et al., 2023）。它是离散扩散的一种具体形式，通过根据上下文迭代预测被掩码的 token，在离散序列上运行。

近期工作已扩大 MDM 的规模（Nie et al., 2026; Ye et al., 2025），其性能可媲美 Llama 3 8B 等相近规模的 ARM（Grattafiori et al., 2024）。它们的

Chinese translation | original equations, data and references retained

双向设计有助于缓解 ARM 特有的局限，例如逆向诅咒（Berglund et al., 2024）；向多模态（Yang et al., 2026; You et al., 2025）和推理任务（Zhao et al., 2026; Huang et al., 2026; Zhu et al., 2025）的扩展，也进一步凸显其作为基础模型范式的通用性。

2.2. 通过缓存机制加速

自回归模型中的键值缓存。语言模型中最成熟的缓存应用是键值（KV）缓存（Pope et al., 2023），它是 ARM 高效率的基础。在 ARM 中，因果注意力允许直接缓存过去 token 的 Key 和 Value 状态，以内存换取计算速度。然而，缓存规模随输入长度增大，形成长上下文部署的瓶颈。为此，已有工作会对已生成的缓存进行稀疏化（Xiao et al., 2024; Zhang et al., 2023; Ge et al., 2024; Liu et al., 2023; Li et al., 2024）。

扩散语言模型中的缓存。虽然 ARM 也有特征缓存研究，但 dLLM 的双向注意力使其不兼容传统 KV 缓存（Nie et al., 2026），构成独特挑战。已有工作开始弥合这一差距，但往往需要缓存感知训练（Arriola et al., 2025），或只能在受限条件下运行（Sahoo et al., 2024）。

同期工作 dKV-Cache（Ma et al., 2026）和 Fast-dLLM（Wu et al., 2026）也研究了利用缓存加速 dLLM。前者侧重解码后的延迟 KV 复用，后者采用块级近似 KV 缓存和置信度感知并行解码。相比之下，dLLM-Cache 利用提示与响应 token 在去噪过程中的非对称动态特征，将长间隔提示缓存与由轻量 V-verify 机制引导的自适应短间隔响应缓存结合。除 KV 状态外，它还缓存注意力输出和 FFN 输出等中间特征，减少 Transformer 块中的重复计算。

3. 方法

3.1. 预备知识

dLLM 的训练范式。与 ARM 顺序、单向的特性不同，dLLM 在去噪框架中训练，学习逆转前向损坏过程；该过程沿连续时间变量随机损坏干净序列。

形式化地，设 $x_0 = (x_1, \cdots, x_L)$ 为从数据分布 D 采样的干净文本序列。前向过程定义连续时间变量 $t \in [0, 1]$：$t = 0$ 表示干净序列，$t = 1$ 表示完全损坏状态。每个时刻 t 都生成一个受损序列 x_t，其中每个 token $x_{i,0}$ 独立地按以下规则变为 $x_{i,t}$：

$$x_{i,t} = \begin{cases} \text{[MASK]} & \text{with probability } t \\ x_{i,0} & \text{with probability } 1 - t \end{cases} \tag{1}$$

这一逐 token 独立掩码过程确保：当 $t \to 1$ 时，序列 x_t 收敛到完全掩码状态。

模型为参数是 θ 的双向 Transformer，记作 p_θ；其训练目标是从受损版本 x_t 重建原始序列 x_0。训练最小化掩码位置上原始 token 的负对数似然。令 M_t 表示 x_t 中被掩码 token 的索引集，损失定义为：

$$\mathcal{L}(\theta) = -\mathbb{E}_{x_0 \sim \mathcal{D}, t \sim U[0,1]} \left[\sum_{i \in \mathcal{M}_t} \log p_\theta(x_{i,0} | x_t) \right] \tag{2}$$

这一训练方式促使模型利用完整的双向上下文学习稳健的语言结构表示，而不受因果依赖链约束。

dLLM 的推理过程。dLLM 通过非自回归过程生成文本，对全掩码序列迭代去噪，得到目标输出。本文重点加速这一推理过程，并以 LLaDA 为代表进行说明。

设 T 为 token 词表，[MASK] ∈ T 为特殊掩码 token。给定提示 $c = (c_1, \cdots, c_M)$，模型经过 K 个离散去噪步生成响应 $y = (y_1, \cdots, y_L)$，步索引 k 从 K 递减至 0。令 $y^{(k)} \in T^L$ 为第 k 步中间状态，初始时是全掩码序列：

$$\mathbf{y}^{(K)} = (\underbrace{\texttt{[MASK]}, \ldots, \texttt{[MASK]}}_{\text{L 个位置}}) \tag{3}$$

在每一步 k，掩码预测器 p_θ 估计干净序列的分布：

$$P_\theta(\mathbf{y} | \mathbf{c}, \mathbf{y}^{(k)}) = p_\theta(\mathbf{c}, \mathbf{y}^{(k)}; \theta) \tag{4}$$

通过贪心解码得到最可能的干净序列估计 $\hat{\mathbf{y}}^{(0|k)}$：

$$\hat{\mathbf{y}}^{(0|k)} = \arg\max_{\mathbf{y} \in \mathcal{T}^L} P_\theta(\mathbf{y} | \mathbf{c}, \mathbf{y}^{(k)}) \tag{5}$$

随后，转移函数 S 根据 $\hat{\mathbf{y}}^{(0|k)}$ 选择性更新 $y^{(k)}$ 中的 token，得到 $y^{(k-1)}$：

$$\mathbf{y}^{(k-1)} = S(\hat{\mathbf{y}}^{(0|k)}, \mathbf{y}^{(k)}, \mathbf{c}, k) \tag{6}$$

S 的具体策略可以是基于置信度的重新掩码，或半自回归块更新（Nie et al., 2026）。虽然该过程支持灵活生成，但跨步骤反复重计算会带来较高延迟，尤其在 K 增大时，详见附录 A.7。

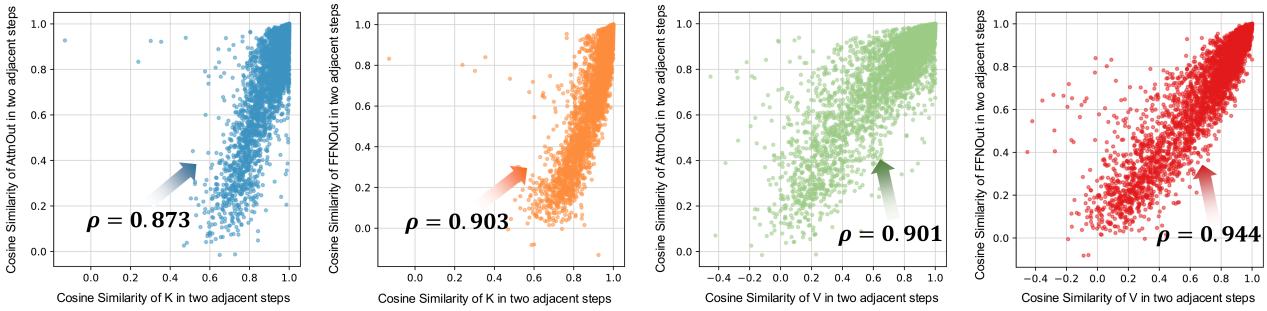
Chinese translation | original equations, data and references retained

原本就存在的两种中间计算结果

Attn：某个 token 从其他 token 聚合信息（多头注意力计算）之后获得的特征向量

v表现稍微比k好一点

FFN就是经过大量非线性变化



(a) Correlation between **K** similarity and **AttnOut** similarity

(b) Correlation between **K** similarity and **FFNOut** similarity

(c) Correlation between **V** similarity and **AttnOut** similarity

(d) Correlation between **V** similarity and **FFNOut** similarity

图 2. 响应 token 在相邻去噪步之间的 K/V 相似度与 AttnOut/FFNOut 相似度的相关性。我们计算每个响应 token 的 K 或 V 向量在当前步与上次缓存步之间的余弦相似度，并将其与对应 AttnOut 或 FFNOut 在同一对步骤间的余弦相似度比较。根据 K/V 相似度选出差异最大的 25% token，绘制两种指标的相关性。子图分别为 K 与 AttnOut（a）、K 与 FFNOut（b）、V 与 AttnOut（c）、V 与 FFNOut（d）的关系。

许多位置的中间向量实际上仍然足够相似，因此可以缓存并复用部分计算结果。

3.2. dLLM-Cache

为缓解 dLLM 的推理低效问题，我们引入无需训练的缓存框架 dLLM-Cache。输入提示在各去噪步保持静态，其内部特征持续稳定，适合长间隔缓存。相比之下，响应序列会动态演化。但这种演化高度稀疏：每一步只有少量响应 token 显著变化。图 1 清楚展示了这种稀疏性，说明通常不必每一步都重算全部响应特征。

为利用这种稀疏性，dLLM-Cache 只选择性更新相邻步骤之间变化最大的少量响应 token。挑战在于如何高效、准确地识别它们。图 2 表明，响应 token 的 Value（V）或 Key（K）向量的变化——以当前版本与缓存版本的余弦相似度量化——与其后续注意力输出（AttnOut）及前馈网络输出（FFNOut）的变化高度相关。这种强相关性说明，监测 V 等较早阶段特征的动态，就能有效识别下游复杂特征也可能发生显著变化的 token。

基于这一发现，我们提出 V-verify 机制。它利用每个响应 token 当前 V 向量与缓存版本的余弦相似度，识别 V 变化最大的 token。只有这些被选中的 token 才进行完整特征重计算与缓存更新。

基于这一核心思想，图 3 展示的 dLLM-Cache 总体流程如下：对每个 Transformer 层 l，将 $K^{(l)}$、$V^{(l)}$、$\mathrm{AttnOut}^{(l)}$ 和 $\mathrm{FFNOut}^{(l)}$ 分别存入提示缓存 C_p 与响应缓存 C_r。缓存由三个超参数控制：

p：prompt

r：response

提示刷新间隔 K_p、响应刷新间隔 K_r 和自适应更新比例 $\rho \in [0, 1]$。推理过程主要包括：

初始化。第一步（$k = K$）根据 $(c, y^{(K)})$ 计算全部特征，将提示相关特征划入 C_p，响应相关特征划入 C_r。

迭代步骤。随后，k 从 $K-1$ 递减至 1，每一层 l 执行：
（1）对于提示，若 $k \equiv 0 \pmod{K_p}$，重新计算并更新 C_p；否则复用。
（2）对于响应，若 $k \equiv 0 \pmod{K_r}$，完整重计算并更新 C_r；否则执行第 3.2.2 节的自适应更新。
（3）各层 l 使用当前可用的特征版本继续前向计算。

终止。在 $k = 0$ 时结束，得到最终输出 $y^{(0)}$。

3.2.1. 提示缓存管理

输入提示 c 不变，因此其特征随时间大体稳定。为利用这一点，dLLM-Cache 维护提示缓存 C_p。$k = K$ 时，计算并存储所有提示相关特征 $K_p^{(l)}$、$V_p^{(l)}$、$\mathrm{AttnOut}_p^{(l)}$ 和 $\mathrm{FFNOut}_p^{(l)}$。后续只每隔 K_p 步重算一次，其他步骤直接从缓存复用。这减少了静态提示的处理成本，尤其在 K_p 较大时。

3.2.2. 带自适应更新的响应缓存

响应特征 $y^{(k)}$ 随时间演化，但大多数 token 在相邻步骤间只缓慢变化，因此可以选择性更新。响应缓存 C_r 支持周期性完整刷新和自适应局部更新两种模式。

Chinese translation | original equations, data and references retained

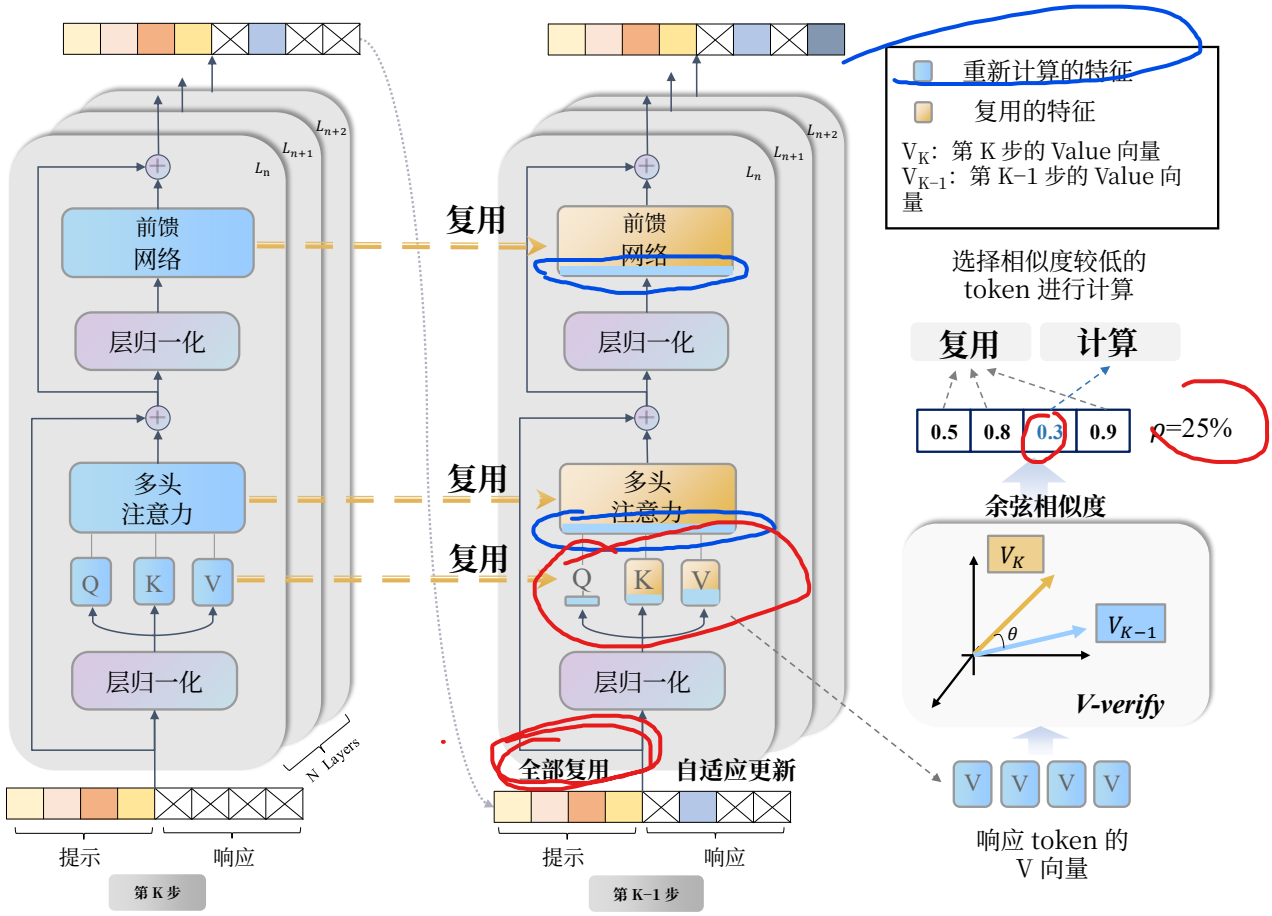


图 3. dLLM-Cache 流程。提示特征按长间隔更新；响应特征每隔 K_r 步完整重算，其余步骤只重算与缓存 V 最不相似的 token。

完整刷新。当 $k \equiv 0 \pmod{K_r}$ 或 $k = K$ 时，重算全部响应特征。

自适应局部更新。否则，先对每个 token j 计算当前 Value 向量 $v_{r,j}^{(l)}$ 与缓存版本 $\tilde{v}_{r,j}^{(l)}$ 的余弦相似度 s_j（式 7）。

为什么是比例？计算开销可以控制

$$s_j = \frac{(\mathbf{v}_{r,j}^{(l)})^\top \tilde{\mathbf{v}}_{r,j}^{(l)}}{\|\mathbf{v}_{r,j}^{(l)}\| \|\tilde{\mathbf{v}}_{r,j}^{(l)}\|} \tag{7}$$

没有设定固定的阈值，而是通过比例去选择

然后选出相似度最低的 $\lfloor \rho |y^{(k)}| \rfloor$ 个 token，重算其特征，其余复用缓存值。在自适应局部更新中，V-verify 计算所有响应 token 的当前 Value 向量，以衡量相邻步骤变化。由于轻量投影后已经得到完整响应侧的 V_r^{new}，我们用它覆盖整个缓存 V_r。相比之下，K、AttnOut 和 FFNOut 仅对选中的低相似度 token 重算，并散写回对应位置。

该自适应策略利用时间稳定性减少计算，同时保持准确率。

4. 实验

4.1. 实验设置

实现细节。我们在两种代表性 dLLM——LLaDA 8B（Nie et al., 2026）和 Dream 7B（Ye et al., 2025）——上评估 dLLM-Cache，各包含 Base 与 Instruct 版本。遵循附录 A.11 中详述的原始推理配置，在八个基准上开展实验。长文本及语义多样场景的额外性能分析见附录 A.1。所有模型均设 ρ = 0.25；K_p、K_r 见附录 A.11。全部实验在 NVIDIA RTX 4090 GPU 上进行。

评估指标。使用多项指标评估 dLLM-Cache 的加速效果与模型质量保持情况。以每秒生成 token 数（TPS）衡量吞吐量、反映推理速度；以每个 token 的平均浮点运算次数（FLOPs）衡量计算成本；以基准分数（例如 GSM8K 准确率，Cobbe et al., 2021）评估任务表现，确保效率收益不会牺牲模型性能。TPS 和 FLOPs 均在单张 RTX 4090 GPU 上测试。

Chinese translation | original equations, data and references retained

flops和实际tps不是完全一致减少，中间会有差距

表 1. LLaDA 8B 在八个基准上使用与不使用 dLLM-Cache 的比较。

任务	方法	推理效率				性能
		TPS↑	加速比(TPS)↑	**FLOPs(T)**↓	加速比(FLOPs)↑	分数↑
数学与科学						
GSM8K	LLaDA Base	7.32	1.00×	16.12	1.00×	69.06
	+ dLLM-Cache	23.19$_{+15.87}$	3.17×$_{+2.17}$	3.21$_{-12.91}$	5.02×$_{+4.02}$	70.66$_{+1.60}$
	LLaDA Instruct	6.95	1.00×	16.97	1.00×	77.48
	+ dLLM-Cache	29.75$_{+22.80}$	4.28×$_{+3.28}$	2.92$_{-14.05}$	5.81×$_{+4.81}$	78.54$_{+1.06}$
GPQA	LLaDA Base	5.12	1.00×	22.97	1.00×	31.91
	+ dLLM-Cache	25.23$_{+20.11}$	4.93×$_{+3.93}$	3.20$_{-19.77}$	7.18×$_{+6.18}$	32.81$_{+0.90}$
	LLaDA Instruct	5.33	1.00×	22.07	1.00×	29.01
	+ dLLM-Cache	28.01$_{+22.68}$	5.26×$_{+4.26}$	2.73$_{-19.34}$	8.08×$_{+7.08}$	29.01$_{+0.00}$
Math	LLaDA Base	8.31	1.00×	14.11	1.00×	30.84
	+ dLLM-Cache	33.92$_{+25.61}$	4.08×$_{+3.08}$	2.61$_{-11.50}$	5.41×$_{+4.41}$	29.84$_{-1.00}$
	LLaDA Instruct	23.65	1.00×	5.16	1.00×	22.32
	+ dLLM-Cache	31.02$_{+7.37}$	1.31×$_{+0.31}$	3.96$_{-1.20}$	1.30×$_{+0.30}$	22.52$_{+0.20}$
通用任务						
MMLU-pro	LLaDA Base	14.08	1.00×	8.40	1.00×	24.26
	+ dLLM-Cache	45.75$_{+31.67}$	3.25×$_{+2.25}$	2.15$_{-6.25}$	3.91×$_{+2.91}$	24.69$_{+0.43}$
	LLaDA Instruct	14.01	1.00×	8.46	1.00×	36.41
	+ dLLM-Cache	39.63$_{+25.62}$	2.83×$_{+1.83}$	2.62$_{-5.84}$	3.23×$_{+2.23}$	36.08$_{-0.33}$
MMLU	LLaDA Base	8.09	1.00×	14.56	1.00×	63.99
	+ dLLM-Cache	33.52$_{+25.43}$	4.14×$_{+3.14}$	2.64$_{-11.92}$	5.52×$_{+4.52}$	64.26$_{+0.27}$
	LLaDA Instruct	10.12	1.00×	11.85	1.00×	61.24
	+ dLLM-Cache	21.23$_{+11.11}$	2.10×$_{+1.10}$	4.50$_{-7.35}$	2.63×$_{+1.63}$	62.82$_{+1.58}$
BBH	LLaDA Base	6.41	1.00×	18.29	1.00×	44.77
	+ dLLM-Cache	27.90$_{+21.49}$	4.35×$_{+3.35}$	3.09$_{-15.20}$	5.92×$_{+4.92}$	45.04$_{+0.27}$
	LLaDA Instruct	6.18	1.00×	18.98	1.00×	51.49
	+ dLLM-Cache	27.55$_{+21.37}$	4.46×$_{+3.46}$	3.08$_{-15.90}$	6.16×$_{+5.16}$	51.98$_{+0.49}$
代码						
MBPP	LLaDA Base	7.87	1.00×	14.91	1.00×	40.80
	+ dLLM-Cache	24.61$_{+16.74}$	3.13×$_{+2.13}$	3.07$_{-11.84}$	4.86×$_{+3.86}$	40.60$_{-0.20}$
	LLaDA Instruct	7.55	1.00×	15.53	1.00×	39.20
	+ dLLM-Cache	31.73$_{+24.18}$	4.20×$_{+3.20}$	2.80$_{-12.73}$	5.55×$_{+4.55}$	39.60$_{+0.40}$
HumanEval	LLaDA Base	19.98	1.00×	6.03	1.00×	32.92
	+ dLLM-Cache	51.96$_{+31.98}$	2.60×$_{+1.60}$	2.04$_{-3.99}$	2.96×$_{+1.96}$	32.31$_{-0.61}$
	LLaDA Instruct	10.57	1.00×	11.10	1.00×	38.71
	+ dLLM-Cache	44.77$_{+34.20}$	4.24×$_{+3.24}$	2.05$_{-9.05}$	5.41×$_{+4.41}$	39.02$_{+0.31}$

4.2. 主要结果

跨模型的性能与效率收益。表 1、表 2 汇总了 LLaDA 8B 和 Dream 7B 的结果。各任务上，dLLM-Cache 均持续提高推理效率，同时基本保留任务性能。对 LLaDA 8B，在 GPQA 上将 dLLM-Cache 应用于 LLaDA Instruct，可取得 8.08 倍 FLOPs 加速，成本从 22.07T 降至 2.73T，准确率不下降。Dream 7B 也呈现相同趋势。这说明 dLLM-Cache 可跨模型系列、跨基准类别泛化，在多种任务上带来广泛的加速。

与其他代表性 LLM 的比较。表 3 比较加速后的 LLaDA 与 Llama 3 8B（Grattafiori et al., 2024）。由于训练数据不同，这仅作为效率参考。单纯将去噪步数减至 32 虽能提高吞吐量，却使准确率灾难性地降至 22.25%。相比之下，dLLM-Cache 在保持 70.66% 准确率的同时实现 2.8 倍加速。再与先进采样方法 SlowFast Sampling（Wei et al., 2026）结合，吞吐量提升至 49.86 TPS，与 Llama 3 8B 相当，同时

Chinese translation | original equations, data and references retained

表 2. Dream 7B 在八个基准上使用与不使用 dLLM-Cache 的比较。

任务	配置	推理效率				性能
		TPS↑	加速比(TPS)↑	**FLOPs(T)**↓	加速比(FLOPs)↑	分数↑
数学与科学						
GSM8K	Dream Base	6.36	1.00×	19.59	1.00×	76.95
	+ dLLM-Cache	32.44$_{+26.08}$	5.10×$_{+4.10}$	2.84$_{-16.75}$	6.90×$_{+5.90}$	76.95$_{+0.00}$
	Dream Instruct	6.39	1.00×	19.57	1.00×	77.55
	+ dLLM-Cache	24.52$_{+18.13}$	3.84×$_{+2.84}$	4.24$_{-15.33}$	4.62×$_{+3.61}$	76.80$_{-0.75}$
GPQA	Dream Base	5.80	1.00×	21.66	1.00×	33.92
	+ dLLM-Cache	30.95$_{+25.15}$	5.33×$_{+4.33}$	3.03$_{-18.63}$	7.15×$_{+6.15}$	34.15$_{+0.23}$
	Dream Instruct	5.53	1.00×	22.63	1.00×	34.38
	+ dLLM-Cache	21.98$_{+16.45}$	3.97×$_{+2.97}$	4.69$_{-17.94}$	4.83×$_{+3.82}$	33.93$_{-0.45}$
Math	Dream Base	9.40	1.00×	13.31	1.00×	38.68
	+ dLLM-Cache	36.89$_{+27.49}$	3.92×$_{+2.92}$	2.61$_{-10.70}$	5.10×$_{+4.10}$	37.94$_{-0.74}$
	Dream Instruct	8.85	1.00×	14.11	1.00×	38.28
	+ dLLM-Cache	23.52$_{+14.67}$	2.66×$_{+1.66}$	4.66$_{-9.45}$	3.03×$_{+2.03}$	37.62$_{-0.66}$
通用任务						
MMLU-pro	Dream Base	15.61	1.00×	7.92	1.00×	24.13
	+ dLLM-Cache	35.86$_{+20.25}$	2.30×$_{+1.30}$	2.89$_{-5.03}$	2.74×$_{+1.74}$	23.86$_{-0.27}$
	Dream Instruct	15.40	1.00×	7.98	1.00×	43.79
	+ dLLM-Cache	23.98$_{+8.58}$	1.56×$_{+0.56}$	4.77$_{-3.21}$	1.67×$_{+0.67}$	43.96$_{+0.17}$
MMLU	Dream Base	9.10	1.00×	13.73	1.00×	73.49
	+ dLLM-Cache	31.07$_{+21.97}$	3.41×$_{+2.41}$	3.27$_{-10.46}$	4.20×$_{+3.20}$	73.20$_{-0.29}$
	Dream Instruct	8.45	1.00×	14.75	1.00×	73.40
	+ dLLM-Cache	38.01$_{+29.56}$	4.50×$_{+3.50}$	2.42$_{-12.33}$	6.10×$_{+5.10}$	73.42$_{+0.02}$
BBH	Dream Base	7.24	1.00×	17.25	1.00×	52.25
	+ dLLM-Cache	29.61$_{+22.37}$	4.09×$_{+3.09}$	3.35$_{-13.90}$	5.15×$_{+4.15}$	51.66$_{-0.59}$
	Dream Instruct	6.98	1.00×	17.90	1.00×	57.07
	+ dLLM-Cache	22.31$_{+15.33}$	3.20×$_{+2.20}$	4.82$_{-13.08}$	3.71×$_{+2.71}$	57.07$_{+0.00}$
代码						
MBPP	Dream Base	8.91	1.00×	14.06	1.00×	54.20
	+ dLLM-Cache	35.69$_{+26.78}$	4.01×$_{+3.01}$	2.66$_{-11.40}$	5.29×$_{+4.29}$	54.20$_{+0.00}$
	Dream Instruct	8.46	1.00×	14.65	1.00×	57.00
	+ dLLM-Cache	29.77$_{+21.31}$	3.52×$_{+2.52}$	3.33$_{-11.32}$	4.40×$_{+3.40}$	56.80$_{-0.20}$
HumanEval	Dream Base	21.43	1.00×	5.68	1.00×	58.53
	+ dLLM-Cache	27.40$_{+5.97}$	1.28×$_{+0.28}$	4.17$_{-1.51}$	1.36×$_{+0.36}$	57.31$_{-1.22}$
	Dream Instruct	17.88	1.00×	6.84	1.00×	57.92
	+ dLLM-Cache	28.03$_{+10.15}$	1.57×$_{+0.57}$	3.94$_{-2.90}$	1.74×$_{+0.74}$	56.09$_{-1.83}$

保持 67.17% 的准确率，比 Llama 3 8B 高 18.12%，体现出我们方法与其他技术的正交性。

表 3. LLaDA 8B Base 与其他代表性 LLM 在 GSM8K 上的比较。

方法	步数	**TPS**↑	准确率 (%)↑	显存 (GB)↓
Llama 3 8B	-	47.73	49.05	16.06
LLaDA Base	32	53.55	22.25	16.94
LLaDA Base	256	7.37	69.06	16.94
+ Cache	256	20.64	**70.66**	17.93
+ Cache + SlowFast	-	49.86	67.17	17.93

与先进采样方法的兼容性。dLLM-Cache 与近期基于采样的加速方法（如 SlowFast Sampling，Wei et al., 2026）相互正交。如表 4 所示，两种方法结合后，在保持模型性能的同时取得更大的推理加速。

与同期缓存方法比较。表 5 比较了 dLLM-Cache 与两项同期工作 dKV-Cache（Ma et al., 2026）、Fast-dLLM（Wu et al., 2026）。它们也旨在加速推理，但主要依赖粗粒度 token 启发式规则，缓存范围局限于 KV 对，往往未优化计算密集的 FFN。相比之下，dLLM-Cache 利用细粒度

FLOPs：模型为了生成一个 token，需要做多少计算
TPS：模型在真实 GPU 上，每秒能够生成多少 token

表 4. LLaDA Base 结合 dLLM-Cache 与 SlowFast Sampling 的性能。

任务	方法	TPS↑	加速比↑	分数↑
数学与科学				
GSM8K	LLaDA Base	4.55	1.00×	69.83
	Sampling + Cache	**26.99**	**5.93×**	**69.60**
GPQA	LLaDA Base	3.31	1.00×	31.47
	Sampling + Cache	**29.06**	**8.78×**	**33.48**
通用任务				
MMLU-pro	LLaDA Base	9.16	1.00×	23.30
	Sampling + Cache	**33.38**	**3.64×**	**25.53**
BBH	LLaDA Base	4.04	1.00×	44.97
	Sampling + Cache	**36.04**	**8.92×**	**44.81**
代码				
HumanEval	LLaDA Base	11.24	1.00×	31.71
	Sampling + Cache	**41.14**	**3.66×**	**31.10**

表 5. dLLM-Cache 与其他同期方法在 LLaDA、Dream 上的比较。

任务	方法	TPS↑	加速比↑	分数↑
GPQA	Dream Base	5.80	1.00×	33.92
	+ dKV-Cache	10.11	1.74×	32.83
	+ Fast-dLLM	22.23	3.83×	31.31
	+ dLLM-Cache	**30.95**	**5.33×**	**34.15**
MMLU	LLaDA Instruct	10.12	1.00×	61.24
	+ dKV-Cache	14.34	1.42×	60.87
	+ Fast-dLLM	20.51	2.03×	61.43
	+ dLLM-Cache	**21.23**	**2.10×**	**62.82**
HumanEval	LLaDA Instruct	10.57	1.00×	38.71
	+ dKV-Cache	14.40	1.36×	37.20
	+ Fast-dLLM	21.50	2.03×	36.59
	+ dLLM-Cache	**44.77**	**4.24×**	**39.02**

V-verify 机制识别特征稳定性，从而可安全跳过整个 Transformer 层的计算，包括注意力与 FFN 投影。这一优势直接转化为更高效率：在 Dream Base 的 GPQA 任务上，dLLM-Cache 实现 5.33 倍加速，显著超过基线的 1.74 倍和 3.83 倍，同时保持最高的 34.15% 准确率。更详细的比较见附录 A.2。

4.3. 消融研究

更新比例 ρ 与选择策略的影响。我们研究了不同自适应更新比例 ρ 下，token 选择策略如何影响性能。图 4 报告 GSM8K 上三种策略的准确率与计算成本：V-verify、K-verify 和随机选择。在广泛的 ρ 取值下，两种基于相似度的策略均持续优于随机选择，验证了动态、特征驱动更新的重要性。尤其是基于 Value 的选择，在 $\rho = 0.25$ 附近取得最高准确率，所需 FLOPs 又远低于完整重计算。这表明适度而有针对性的更新（例如 $\rho \approx 0.25$）能在效率与输出质量间取得良好折中。

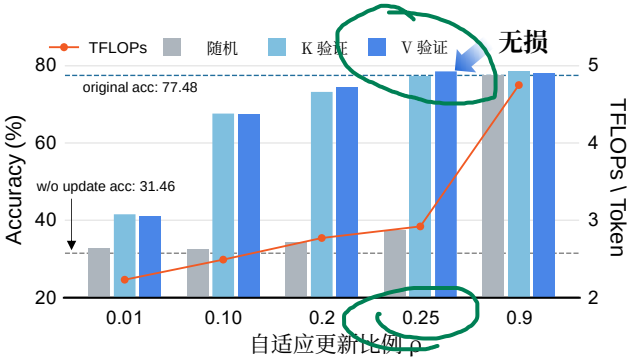


图 4. 不同更新比例 ρ 下，token 选择策略对 LLaDA 8B Instruct 在 GSM8K 上的影响。

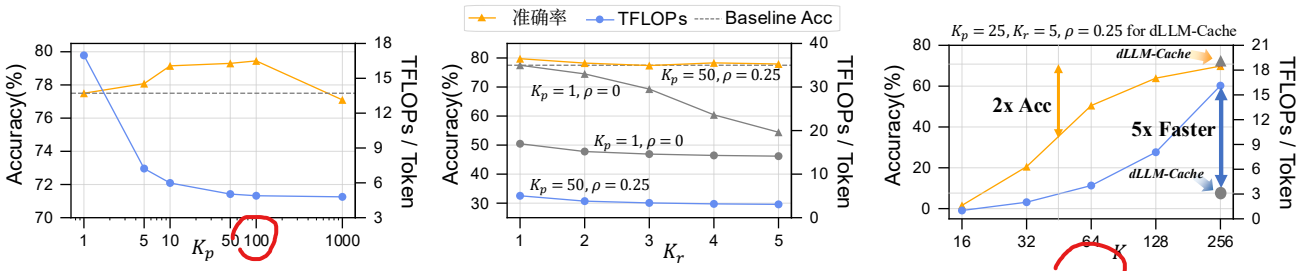
缓存刷新间隔 K_p 与 K_r 的影响。我们分析刷新间隔对效率和准确率的影响。图 5(a) 显示，增大提示间隔 K_p 能显著减少 FLOPs 而不损害准确率，说明无需频繁更新提示。图 5(b) 突出了响应更新的重要性：不使用提示缓存或自适应更新时（$K_p = 1$、$\rho = 0$，灰线），准确率急降；我们的设置（$K_p = 50$、$\rho = 0.25$，橙线和蓝线）则以低得多的成本保持高准确率。这验证了长提示间隔、短响应间隔和自适应更新结合的策略。Dream 模型的额外分析见附录 A.3。

5. 讨论

去噪步数的影响。dLLM 的去噪步数决定质量与延迟之间的折中。如图 5(c) 所示，增加步数提高输出准确率，也增加推理成本。单纯减少步数虽能加速，却导致严重性能退化。在 GSM8K 上，dLLM-Cache 在 256 步时实现 5 倍无损加速，计算成本相当于约 48 步的基线，准确率却是其两倍以上。这说明与单纯减少步数不同，我们的方法兼顾效率与质量。

缓存存储开销。dLLM-Cache 在每一层存储四类中间特征：K、V、AttnOut 和 FFNOut。总缓存规模随 token 数 T、嵌入维度 d 和层数 L 增长，开销为 $T \times d \times 4 \times L$，详见附录 A.9。

Chinese translation | original equations, data and references retained



(a) Effect of K_p on GSM8K (b) Effect of K_r on GSM8K (c) Effect of K on GSM8K

图 5.（a）在 $K_r = 1$、$\rho = 0$ 时改变 K_p。（b）在基线（$K_p = 1$、$\rho = 0$，灰色）和本文设置（表 1 中 $K_p = 50$、$\rho = 0.25$）下改变 K_r。（c）改变无缓存基线的去噪步数 K；灰色标记为 $K_p = 25$、$K_r = 5$、$\rho = 0.25$ 的 dLLM-Cache。其帕累托折中更优：相近准确率下比标准 256 步基线快约 5 倍；相同 FLOPs 下准确率超过基线两倍。（a–b）使用 LLaDA Instruct；（c）使用 LLaDA Base。

由于每层只缓存一个版本，整体占用保持稳定。如表 3 所示，在 GSM8K 上，LLaDA 8B Base 无缓存时的 GPU 峰值占用为 16.94 GB，使用 dLLM-Cache 时为 17.93 GB，Llama 3 8B 为 16.06 GB。因此，缓存增加的显存不到 1 GB，相对于基线仅增加 5%。这点存储开销换来最高 9 倍生成加速，是十分有利的折中，尤其适用于延迟要求严格、显存预算有限的场景。

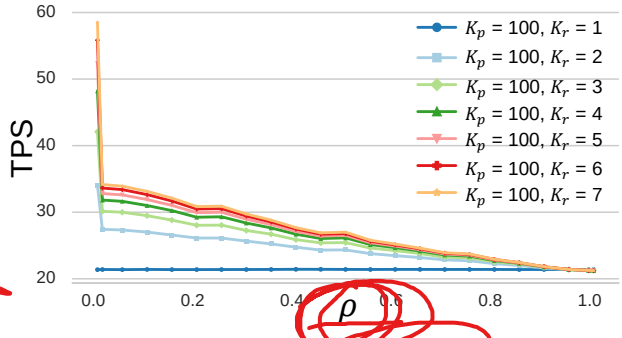


图 6. TPS 与 ρ 的关系。极小 ρ 处 TPS 明显下降，反映启动选择性更新的固定成本。

V-verify 的成本与固定更新开销。 V-verify 利用轻量 V 向量相似度识别动态 token。尽管其本身计算开销低，自适应局部更新的实际加速仍受固定操作开销限制。图 6 显示，从完全缓存状态转为自适应更新（$\rho > 0$）时存在延迟代价。原因是任何选择性重计算都会触发不可忽略的系统级延迟，例如 GPU 内核管理和数据移动；这些开销并不与更新 token 数严格成比例。因此，在 ρ 很小时，固定开销占据主导，限制运行时间进一步减少。最优 ρ 必须在保持模型质量的同时，平衡固定成本与动态计算节省。图 4 表明，$\rho \approx 0.25$ 能在启动选择性更新的成本与计算减少的收益间取得有效折中，从而优化整体效率和保真度。

6. 结论

我们提出 dLLM-Cache，一种无需训练、与模型无关的缓存方法，用于加速基于扩散的大语言模型推理。在 LLaDA 和 Dream 上的广泛评测证实，dLLM-Cache 在保持有竞争力的生成质量的同时，最多将 FLOPs 减少至原来的 1/9.1。

影响声明

本文工作的目标是推动机器学习领域的发展。该工作可能带来多种潜在社会影响，但我们认为没有需要在此特别强调的事项。

致谢

本工作得到 CCF—腾讯犀牛鸟基金资助。

参考文献

Arriola, M., Gokaslan, A., Chiu, J. T., Yang, Z., Qi, Z., Han, J., Sahoo, S. S., and Kuleshov, V. Block diffusion: Interpolating between autoregressive and diffusion language models. 2025. URL https://openreview.net/forum?id=tyEyYT267x.

Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and Van Den Berg, R. Structured denoising diffusion models in discrete state-spaces. *Advances in Neural Information Processing Systems*, 34:17981–17993, 2021.

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and

Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, 2024.

Berglund, L., Tong, M., Kaufmann, M., Balesni, M., Stickland, A. C., Korbak, T., and Evans, O. The reversal curse: LLMs trained on "a is b" fail to learn "b is a". In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=GPKTIktA0k.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901, 2020.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=uNrFpDPMyo.

Gong, S., Agarwal, S., Zhang, Y., Ye, J., Zheng, L., Li, M., An, C., Zhao, P., Bi, W., Han, J., Peng, H., and Kong, L. Scaling diffusion language models via adaptation from autoregressive models. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=j1tSLYKwg8.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

He, Z., Sun, T., Tang, Q., Wang, K., Huang, X., and Qiu, X. DiffusionBERT: Improving generative masked language models with diffusion models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 4521–4534, July 2023.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Huang, Z., Chen, Z., Wang, Z., Li, T., and Qi, G.-J. Reinforcing the diffusion chain of lateral thought with diffusion language models. *Advances in Neural Information Processing Systems*, 38:152677–152710, 2026.

Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.

Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., and Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36:52342–52364, 2023.

Lou, A., Meng, C., and Ermon, S. Discrete diffusion modeling by estimating the ratios of the data distribution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 32819–32848, 2024.

Ma, X., Yu, R., Fang, G., and Wang, X. dkv-cache: The cache for diffusion language models. *Advances in Neural Information Processing Systems*, 38:149009–149033, 2026.

Nie, S., Zhu, F., Du, C., Pang, T., Liu, Q., Zeng, G., Lin, M., and Li, C. Scaling up masked diffusion models on text. 2025. URL https://openreview.net/forum?id=WNvvwK0tut.

Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., Zhou, J., Lin, Y., Wen, J.-R., and Li, C. Large language diffusion models. *Advances in Neural Information Processing Systems*, 38:50608–50646, 2026.

OpenAI. ChatGPT: Optimizing Language Models for Dialogue. *OpenAI blog*, November 2022. URL https://openai.com/blog/chatgpt/.

Ou, J., Nie, S., Xue, K., Zhu, F., Sun, J., Li, Z., and Li, C. Your absorbing discrete diffusion secretly models the conditional distributions of clean data. 2025. URL https://openreview.net/forum?id=sMyXP8Tanm.

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4195–4205, 2023.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*, volume 5, pp. 606–624, 2023.

Radford, A., Narasimhan, K., Salimans, T., and Sutskever, I. Improving language understanding by generative pre-training, 2018.

Chinese translation | original equations, data and references retained

Reid, M., Hellendoorn, V. J., and Neubig, G. DiffusER: Diffusion via edit-based reconstruction. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=nG9RF9z1yy3.

Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10684–10695, 2022.

Sahoo, S. S., Arriola, M., Schiff, Y., Gokaslan, A., Marroquin, E., Chiu, J. T., Rush, A., and Kuleshov, V. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, volume 37, pp. 130136–130184, 2024.

Shi, J., Han, K., Wang, Z., Doucet, A., and Titsias, M. Simplified and generalized masked diffusion for discrete data. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=xcqSOfHt4g.

Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pp. 2256–2265. PMLR, 2015.

Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

Wei, Q., Zhang, Y., Liu, Z., Zeng, P., Wang, Y., Qi, B., Liu, D., and Zhang, L. Accelerating diffusion large language models with slowfast sampling: The three golden principles. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=Uh17FiwF4q.

Wu, C., Zhang, H., Xue, S., Liu, Z., Diao, S., Zhu, L., Luo, P., Han, S., and Xie, E. Fast-dLLM: Training-free acceleration of diffusion LLM by enabling KV cache and parallel decoding. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=3Z3Is6hnOT.

Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.

Yang, L., Tian, Y., Li, B., Zhang, X., Shen, K., Tong, Y., and Wang, M. Mmada: Multimodal large diffusion language models. *Advances in Neural Information Processing Systems*, 38:138867–138907, 2026.

Ye, J., Zheng, Z., Bao, Y., Qian, L., and Gu, Q. Diffusion language models can perform many tasks with scaling and instruction-finetuning. *arXiv preprint arXiv:2308.12219*, 2023.

Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., and Kong, L. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

You, Z., Nie, S., Zhang, X., Hu, J., Zhou, J., Lu, Z., Wen, J.-R., and Li, C. Llada-v: Large language diffusion models with visual instruction tuning. *arXiv preprint arXiv:2505.16933*, 2025.

Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36, 2023.

Zhao, S., Gupta, D., Zheng, Q., and Grover, A. d1: Scaling reasoning in diffusion large language models via reinforcement learning. *Advances in Neural Information Processing Systems*, 38:56729–56762, 2026.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

Zheng, L., Yuan, J., Yu, L., and Kong, L. A reparameterized discrete diffusion model for text generation. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=PEQFHRUFca.

Zhu, F., Wang, R., Nie, S., Zhang, X., Wu, C., Hu, J., Zhou, J., Chen, J., Lin, Y., Wen, J.-R., et al. Llada 1.5: Variance-reduced preference optimization for large language diffusion models. *arXiv preprint arXiv:2505.19223*, 2025.

A. 附录

A.1. 长文本与语义多样场景中的性能

长输入的效率。长间隔提示缓存机制对包含长静态提示的场景尤为有利。例如，在 LongBench-HotpotQA（Bai et al., 2024）上使用 LLaDA 8B Base 时，dLLM-Cache 相较基线将 FLOPs 减少至 1/9.1。值得注意的是，加速还伴随性能提升：F1 分数从 34.56 升至 36.10，说明我们的方法能够充分利用提示稳定性而不牺牲精度。

表 6. LLaDA Instruct 与 Dream Instruct 在 LongBench 上使用或不使用 dLLM-Cache 的性能比较。

方法	单文档问答		多文档问答			摘要			少样本学习			合成任务	代码		平均分数
	Qasper	*MF-en*	*HotpotQA*	*2WikiMQA*	*Musique*	*GovReport*	*QMSum*	*MultiNews*	*TREC*	*TriviaQA*	*SAMSum*	*PRe*	*Lcc*	*RB-P*	
LLaDA Instruct	16.96	31.31	14.68	17.60	11.48	29.24	21.93	27.58	65.20	47.98	40.51	98.17	65.69	59.57	**39.14**
+ dLLM-Cache	15.26	29.62	13.87	17.17	10.44	29.75	22.06	26.68	66.00	44.94	41.86	97.44	66.07	59.34	**38.61**
Dream Instruct	28.17	36.23	27.65	32.43	11.83	5.04	14.29	5.95	73.00	89.25	37.84	16.92	38.91	45.08	**33.04**
+ dLLM-Cache	26.55	39.86	27.66	32.09	11.12	4.40	13.89	5.51	73.50	89.59	36.07	12.05	39.88	45.57	**32.70**

多样任务上的稳健性。为验证这种效率提升不会损害复杂真实场景中的通用能力，我们使用 Instruct 模型，在综合性的 LongBench 基准上评估 dLLM-Cache。表 6 列出六大任务类别的性能。dLLM-Cache 显示出较强的性能保持能力：LLaDA Instruct 的平均分为 38.61，基线为 39.14；Dream Instruct 为 32.70，基线为 33.04。结果证实，dLLM-Cache 能在广泛任务中有效保留深层语义理解与长距离依赖推理能力。

A.2. 与同期缓存方法的详细比较

表 5 提供简要概览；表 7 进一步全面比较 dLLM-Cache 与同期缓存方法 dKV-Cache（Ma et al., 2026）、Fast-dLLM（Wu et al., 2026）。

表 7. dLLM-Cache 与其他同期方法在 LLaDA（左）和 Dream（右）上的比较。

任务	方法	TPS↑	加速比↑	显存↓	分数↑
GSM8K	LLaDA Instruct	6.95	1.00×	15.86	77.48
	+ dKV-Cache	8.89	1.28×	21.08	**79.30**
	+ Fast-dLLM	19.11	2.75×	19.48	75.89
	+ dLLM-Cache	**29.75**	**4.28**×	**17.85**	78.54
MMLU	LLaDA Instruct	10.12	1.00×	15.54	61.24
	+ dKV-Cache	14.34	1.42×	17.88	60.87
	+ Fast-dLLM	20.51	2.03×	17.13	61.43
	+ dLLM-Cache	**21.23**	**2.10**×	**16.61**	**62.82**
HumanEval	LLaDA Instruct	10.57	1.00×	15.39	38.71
	+ dKV-Cache	14.40	1.36×	17.17	37.20
	+ Fast-dLLM	21.50	2.03×	**16.60**	36.59
	+ dLLM-Cache	**44.77**	**4.24**×	16.65	**39.02**

任务	方法	TPS↑	加速比↑	显存↓	分数↑
GSM8K	Dream Base	6.36	1.00×	15.73	76.95
	+ dKV-Cache	10.26	1.61×	**16.14**	76.57
	+ Fast-dLLM	21.36	2.08×	19.95	74.30
	+ dLLM-Cache	**32.44**	**5.10**×	16.76	**76.95**
GPQA	Dream Base	5.80	1.00×	15.77	33.92
	+ dKV-Cache	10.11	1.74×	**16.23**	32.83
	+ Fast-dLLM	22.23	3.83×	20.69	31.31
	+ dLLM-Cache	**30.95**	**5.33**×	16.93	**34.15**
MMLU	Dream Base	9.10	1.00×	15.64	73.49
	+ dKV-Cache	12.80	1.41×	**15.92**	72.77
	+ Fast-dLLM	23.69	2.60×	18.32	72.69
	+ dLLM-Cache	**31.07**	**3.41**×	16.37	**73.20**

A.3. Dream 7B 上的详细敏感性分析

正文表明，dLLM-Cache 对 LLaDA、Dream 等不同 dLLM 架构均有效。这体现了方法的泛化性：它针对扩散过程固有的计算冗余，而非特定模型的偶然特征。

为进一步支持方法的稳健性并深入理解其行为，本节详细分析 dLLM-Cache 应用于 Dream 7B 时关键超参数的敏感性。结果

Chinese translation | original equations, data and references retained

见表 8、表 9 和表 10；其中的性能趋势与 LLaDA 上的观察高度一致，证实了方法在不同模型间稳定、可预测的行为。

表 8. Dream 7B 在 GPQA 上对自适应更新比例 ρ 的敏感性分析。设置 $K_p = 25$、$K_r = 4$。

ρ	0	0.1	0.2	0.25	0.3	0.5	0.75	1
准确率 (%)	35.04	36.16	35.93	35.04	35.04	34.59	35.49	35.26

表 9. Dream 7B 在 GPQA 上对提示刷新间隔 K_p 的敏感性分析。设置 $K_r = 4$、$\rho = 0.25$。

K_p	10	25	50	100
准确率 (%)	35.04	35.04	35.04	35.04

表 10. Dream 7B 在 GPQA 上对响应刷新间隔 K_r 的敏感性分析。设置 $K_p = 25$、$\rho = 0.25$。

K_r	2	4	6
准确率 (%)	36.16	35.04	33.92

A.4. Token 选择的跨层行为

dLLM-Cache 的逐层 token 选择是其缓存设计的自然结果。每个 Transformer 层 l 都独立存储缓存特征：在提示缓存 C_p 和响应缓存 C_r 中保存 $K^{(l)}$、$V^{(l)}$、$\mathrm{AttnOut}^{(l)}$、$\mathrm{FFNOut}^{(l)}$。因此，V-verify 分数通过比较当前 Value 向量与缓存版本，在第 l 层局部定义。这使逐层 token 选择成为与该缓存设计最一致的方案。

选中的 token 集合预计在相邻层之间大体稳定。设 $s_j^{(l)}$ 为响应 token j 在第 l 层的 V-verify 分数。在标准平滑性假设下，若层映射与 Value 投影均满足 Lipschitz 条件，则存在较小的 ε，使 $|s_j^{(l)} - s_j^{(l+1)}| \leqslant \varepsilon$。令 Δ 表示选择边界处的间隔，$\Delta = s_{(\lfloor \rho |y| \rfloor)} - s_{(\lfloor \rho |y| \rfloor + 1)}$。若 $\varepsilon < \Delta$，则从第 l 层到第 l+1 层，最低 $\lfloor \rho |y| \rfloor$ 个元素的归属保持不变。因此，除非大量 token 集中在截断点附近，选中集合应随深度保持大体稳定。

为研究不同层是否受益于不同更新比例，我们在 GSM8K 上使用 LLaDA Instruct，设置 $K_p = 50$、$K_r = 7$，比较平均更新预算同为 0.25 的三种配置。表 11 显示，全局固定 $\rho = 0.25$ 的准确率最高，优于偏重低层更新和偏重高层更新的分配。虽然低层更新可通过后续 Transformer 计算传播到高层特征，但在总体更新预算相同的条件下，这一影响不足以要求随深度变化的更新比例。

表 11. LLaDA Instruct 在 GSM8K 上的跨层更新比例分配。所有策略的平均预算均为 0.25。

策略（平均预算 = 0.25）	GSM8K 准确率 (%)
低层更新更多（0.375 → 0.125）	78.70
高层更新更多（0.125 → 0.375）	79.23
全局固定（$\rho = 0.25$）	**79.83**

A.5. 完整覆盖 V 缓存与选择性更新之间的设计选择

如第 3.2.2 节所述，自适应局部更新时，V-verify 计算所有响应 token 的当前 Value 向量，以衡量相邻步变化。由于轻量投影后已获得整个响应侧的 V_r^{new}，

Chinese translation | original equations, data and references retained

我们用 V_r^{new} 覆盖整个缓存 V_r，而非只更新选中的低相似度 token 的 Value 向量。这是 V-verify 机制针对 Value 向量作出的有意设计选择。

为验证该设计，我们在 GSM8K 上使用 LLaDA 8B Instruct，将当前方法与仅更新选中 token 的 Value 向量的变体比较。完整覆盖取得 79.83% 准确率，高于仅更新选中 token 的 78.85%。结果支持当前设计，并证实：用最新 Value 向量覆盖完整 V 缓存，为下一步相似度检查提供了更好的基础。

A.6. 相似度度量的影响

我们比较了余弦相似度与 L2 距离作为 V-verify 相似度度量的效果。在 GSM8K 的 LLaDA 8B Instruct 上，余弦相似度取得 78.54% 准确率，显著优于 L2 距离的 55.95%。这表明余弦相似度更能捕捉语义变化，因此整个方法均默认采用它。

A.7. 复杂度与延迟分析

本节详细分析原始 dLLM 推理过程及所提出 dLLM-Cache 框架的计算复杂度。

原始 dLLM 模型的复杂度。LLaDA、Dream 等标准 dLLM 采用具有双向注意力的多层 Transformer 架构。从全掩码序列出发，通过 K 个迭代去噪步生成文本。每一步都对长度为 n 的完整输入序列进行完整前向传播。以 FLOPs 衡量的每步计算成本主要由注意力和前馈网络（FFN）层决定：

$$\text{FLOPs}_{\text{step}} = T \cdot (8nd^2 + 4n^2d + 6ndm) \tag{8}$$

其中 T 为 Transformer 层数，n 为序列长度，d 为隐藏维度，m 为 FFN 中间维度。注意，对 LLaDA 等使用 SwiGLU 的架构，FFN 包含三次矩阵乘法（门控、上投影、下投影），贡献 6ndm FLOPs。

因此，标准 dLLM 的总推理复杂度为每步成本乘以步数 K：

$$\text{FLOPs}_{\text{dLLM}} = K \cdot T \cdot (8nd^2 + 4n^2d + 6ndm) \tag{9}$$

使用 dLLM-Cache 后的复杂度。dLLM-Cache 缓存中间状态并只选择性更新部分 token，将计算分为完整刷新、仅响应刷新和自适应局部更新三类。总复杂度近似为：

$$\begin{aligned}\text{FLOPs}_{\text{dLLM-Cache}} \approx\ & \frac{K}{K_p} \cdot T \cdot (8nd^2 + 4n^2d + 6ndm) \\ & + \left(\frac{K}{K_r} - \frac{K}{K_p}\right) \cdot T \cdot (8rd^2 + 4rnd + 6rdm) \\ & + K \cdot \left(1 - \frac{1}{K_r}\right) \cdot T \cdot (2rd^2 + 8\hat{r}d^2 + 4\hat{r}nd + 6\hat{r}dm)\end{aligned} \tag{10}$$

其中 K_p、K_r 分别是提示、响应的刷新间隔；p、r 分别为提示和响应长度（n = p + r）；$\hat{r} = \rho \cdot r$ 为自适应步骤中更新的响应 token 数，ρ 是自适应更新比例。

计算节省。上述公式是三种不同运行模式的加权和。第一项 $(K/K_p) \cdot T \cdot (\cdots)$ 表示周期性完整刷新的成本。虽然每次成本与原始 dLLM 相同，但发生频率较低，由较大的 K_p（通常 $\geqslant 100$）控制。

第二项 $(K/K_r - K/K_p) \cdot T \cdot (8rd^2 + 4rnd + 6rdm)$ 衡量更频繁的“仅响应刷新”的成本。其收益很明显：二次注意力项由 $O(n^2)$ 降至 $O(rn)$，因为只为响应 token（长度 r）计算新的 Query 向量，使其关注完整序列（长度 n）。这一项代表折中模式：定期更新演化中的响应，同时利用提示的静态缓存。

Chinese translation | original equations, data and references retained

第三项，即最后一项 $K \cdot (1 - 1/K_r) \cdot T \cdot (2rd^2 + 8\hat{r}d^2 + 4\hat{r}nd + 6\hat{r}dm)$，是效率提升的关键。它描述了占绝大多数去噪步的自适应局部更新，其中包含 V-verify 为全部响应 token 计算 Value 向量所需的 $2rd^2$ 轻量开销。关键在于，相较将 r 个 token 的完整计算替换为 $\hat{r} = \rho \cdot r$ 所定义的稀疏子集带来的大量节省，这点固定成本可忽略。ρ 通常设为 25%，显著减轻二次注意力和计算繁重的 FFN 层的负担。因此，我们以极小验证成本换取主要计算负载的大幅减少。

dLLM-Cache 的加速主要来自同时对注意力与 FFN 层应用稀疏更新。具体地，将昂贵的全序列运算换为轻量局部更新：二次注意力项从 $4n^2d$ 降至 $4\hat{r}nd$，繁重的 FFN 计算从 $6ndm$ 降至 $6\hat{r}dm$。相对计算节省可写为：

$$\text{Savings} = 1 - \frac{\text{FLOPs}_{\text{dLLM-Cache}}}{\text{FLOPs}_{\text{dLLM}}} \tag{11}$$

实验表明，计算需求的显著减少可大幅提高推理速度，在实际场景中最多将 FLOPs 减少至原来的 1/9.1。

A.8. 理论误差界分析

为 Transformer 等复杂迭代系统推导严格、闭式的误差界，仍是开放且困难的问题。本节对 dLLM-Cache 引入的近似误差进行理论分析，表明它可在合理假设下有界，从而解释支撑其实证稳定性的机制。

误差传播形式化。首先形式化误差来源。设 y_k 为真实隐藏状态轨迹，$\tilde{y}_k$ 为近似轨迹，第 k 步误差定义为 $\delta_k = y_k - \tilde{y}_k$。误差沿 Transformer 网络 F 传播，假设 F 满足 Lipschitz 连续性，常数为 L_F。去噪更新可抽象为：

$$y_{k-1} \approx \alpha_k y_k + (1 - \alpha_k) F(y_k) \tag{12}$$

于是，下一步误差 δ_{k-1} 的界为：

$$\begin{aligned}
\|\delta_{k-1}\| &= \|(\alpha_k y_k + (1-\alpha_k)F(y_k)) - (\alpha_k \tilde{y}_k + (1-\alpha_k)F(\tilde{y}_k))\| \\
&\le \alpha_k \|\delta_k\| + (1-\alpha_k)\|F(y_k) - F(\tilde{y}_k)\| \\
&\le \alpha_k \|\delta_k\| + (1-\alpha_k) L_F \|\delta_k\| \\
&= (\alpha_k + (1-\alpha_k) L_F)\|\delta_k\|
\end{aligned}$$

令 $C_k = \alpha_k + (1 - \alpha_k)L_F$，有 $\| \delta_{k-1} \| \leqslant C_k \| \delta_k \|$。由于表达能力强的模型满足 $L_F > 1$，因此 $C_k > 1$，说明在没有干预时误差自然倾向于指数放大。

双机制误差控制。这正是 dLLM-Cache 双机制设计的关键。第一种机制是由响应刷新间隔 K_r 强制实施的周期性误差重置。每个刷新步 k_0（满足 $k_0 \bmod K_r = 0$）都使误差有效重置，即 $\| \delta_{k_0} \| \approx 0$，保证误差积累局限在长度为 K_r 的有限窗口内。若从 k_0 起连续 j 步不控制误差，其量级大致为：

$$\|\delta_{k_0 - j}\| \le \left(\prod_{i=1}^{j} C_{k_0 - i + 1} \right) \|\delta_{k_0}\| \tag{13}$$

重置机制阻止 j 无限增长，从而保证整体误差有界。

为进一步控制这些区间内的误差，dLLM-Cache 使用 V-verify 自适应更新。每个非刷新步更新比例为 ρ 的 token，意味着只有缓存的 $(1 - \rho)$ 部分 token 对应误差被放大；将该部分投影记作 P_{cache}。更准确的误差递推为：

$$\|\delta_{k-1}\| \le C_k \|P_{cache}(\delta_k)\| + \epsilon_{\text{step}} \tag{14}$$

Chinese translation | original equations, data and references retained

其中 $\varepsilon_{\text{step}}$ 是新计算 token 引入的小误差。由于 $\| P_{\text{cache}}(\delta_k) \|$ 严格小于 $\| \delta_k \|$，有效误差放大系数也更小，即 $C'_k < C_k$。因此，刷新窗口内最大累积误差显著减少：峰值不再为 $O(C^{K_r})$ 量级，而是 $O((C')^{K_r})$，得到更紧的界。

结论及与实验的一致性。综上，数学分析表明 dLLM-Cache 的近似误差稳定有界。上界 $\sup_k|\delta_k|$ 通过 C_k 依赖模型性质，也依赖刷新间隔 K_r 及决定 C'_k 的自适应更新比例 ρ。周期性重置 K_r 防止误差无限积累，为有界性提供硬保证；V-verify 则通过几乎每一步都主动抑制误差增长，显著收紧该界。

这一理论发现与图 5(b) 的实验观察完全一致：不使用 V-verify（$\rho = 0$）时，准确率随 K_r 增大而显著下降。相反，启用 V-verify 后，即使只更新 25% 的 token（$\rho = 0.25$），在能大量节省计算的较大 K_r 下，仍保持与基线几乎相同的准确率。这表明，在不牺牲高保真度的前提下实现显著加速，得益于受到良好控制的误差。

A.9. 缓存存储开销的证明

定理：本方法的缓存存储开销为 $O(T \times d \times 4 \times L)$，其中 T 是 token 数，d 是嵌入维度，L 是层数。

证明。每个 Transformer 层都缓存四种中间特征张量：K、V、AttnOut 和 FFNOut。每个张量大小为 $T \times d$，因此每层缓存大小为

$$M_{\text{layer}} = 4 \times T \times d$$

跨 L 层，总缓存包含

$$M_{\text{total}} = L \times M_{\text{layer}} = L \times 4 \times T \times d$$

个元素。使用 bfloat16 存储时，相当于 $2 \times 4 \times L \times T \times d$ 字节。因此，存储开销为：

$$O(T \times d \times 4 \times L)$$

证毕。 □

A.10. 局限性

尽管 dLLM-Cache 显著加速并保持有竞争力的生成质量，它仍有一些局限，也为后续工作提供机会。（1）**长序列的显存开销**。缺乏 PagedAttention 等内存管理系统时，相比标准 dLLM 推理，缓存中间特征会增加长上下文场景的内存开销。近期工作开始处理 dLLM 长上下文中的内存与驱逐问题，将这些技术与 dLLM-Cache 整合是有前景的方向。（2）**近似性质**。与 ARM 中精确的 KV 缓存不同，dLLM-Cache 以少量准确率换取较大的速度收益，是近似技术。实验表明质量损失通常较小，但它在根本上并非无损。我们希望坦诚说明这些局限，以指引未来在 dLLM-Cache 基础上的改进。

A.11. 实验细节

本节汇总所评测 dLLM 的 Base 和 Instruct 版本的实验配置。针对每个任务，报告去噪步数、块长度、生成长度、重新掩码策略、少样本设置、提示刷新间隔 K_p 和响应刷新间隔 K_r。除非另有说明，所有模型均使用低置信度重新掩码。基线与 dLLM-Cache 均在相同的 PyTorch 和 Hugging Face Transformers 框架下评估。表 1、表 2 的批大小为 8，表 3 为 1。

Chinese translation | original equations, data and references retained

K_p 和 K_r 可按任务需求灵活调整，而非通过超参数调优确定。代码生成、数学推理等准确率敏感任务可采用较小间隔；延迟为主要考量时，较大间隔能减少计算。因此，这些设置控制精度和效率的实际平衡，而不是所观察到加速的来源。

由于采用既有工作（Nie et al., 2026）的基准配置，Base 与 Instruct 模型的收益幅度有时不同。例如，MMLU 对 Base 使用 256-token 生成长度与解码步数，而对 Instruct 仅使用 3。因为本方法的加速随 token 数和去噪步数变化，这会造成不同加速比，详见附录 A.7。

表 12. Instruct 模型在所选基准上的实验设置。

任务	步数	块长度	生成长度	少样本数
GSM8K	256	8	256	4
GPQA	128	64	128	5
Math	256	256	256	0
MMLU-pro	256	256	256	0
MMLU	3	3	3	5
MBPP	512	32	512	3
BBH	256	256	256	3
HumanEval	512	32	512	0

表 13. LLaDA Base 在所选基准上的间隔步数。

	GSM8K	GPQA	Math	MMLU-pro	MMLU	BBH	MBPP	HumanEval	平均
K_p	25	100	50	100	100	50	25	100	69
K_r	5	8	8	6	6	6	4	5	6

表 14. LLaDA Instruct 在所选基准上的间隔步数。

	GSM8K	GPQA	Math	MMLU-pro	MMLU	BBH	MBPP	HumanEval	平均
K_p	50	50	50	51	100	100	100	25	66
K_r	7	6	1	3	7	5	5	5	5

表 15. Dream Base 在所选基准上的间隔步数。

	GSM8K	GPQA	Math	MMLU-pro	MMLU	BBH	MBPP	HumanEval	平均
K_p	100	100	100	25	100	25	25	5	60
K_r	8	8	4	2	2	4	8	1	5

表 16. Dream Instruct 在所选基准上的间隔步数。

	GSM8K	GPQA	Math	MMLU-pro	MMLU	BBH	MBPP	HumanEval	平均
K_p	25	10	50	5	100	10	10	50	33
K_r	2	8	1	1	8	2	8	1	4

Chinese translation | original equations, data and references retained